SESSION XXI

The Discord Gateway Sprint

Autonomous Trade Bot · May 4, 2026

Discord Gateway live · 4/6 signal feeds connected · end-to-end message capture confirmed · boot order hardened · Pydantic Settings extended · production deployment successful

SESSION OVERVIEW

Session XXI completed the Discord signal feed — the final piece of the 6-feed inbound signal architecture. A Discord bot (OTF Signal Bot) was created, configured, and connected to the production backend via discord.py's Gateway WebSocket. A live end-to-end test confirmed messages containing TAO/Bittensor keywords posted in the OTF Signals Discord server appear in the Activity Log as SIGNAL events within seconds.

Additionally, a pre-existing boot-order defect was discovered and resolved: bittensor's substrate-interface was blocking the asyncio event loop on startup, preventing the Discord Gateway and other async tasks from establishing connections. The fix restructures the boot sequence so signal feeds start first and chain-dependent services run as non-blocking tasks.

TASK COMPLETION

#	Task	Status	Commit
1	Discord bot created (OTF Signal Bot #8669)	✓ Done	d0081eda
2	discord.py 2.x Gateway connector + keyword filter	✓ Done	d0081eda
3	requirements.txt: discord.py>=2.3.2	✓ Done	d0081eda
4	Settings: 3 API key fields added to Pydantic model	✓ Done	88779820
5	Boot order: signal feeds before chain I/O	✓ Done	88779820
6	Boot order: bittensor probe removed (blocks event loop)	✓ Done	88779820
7	Boot order: chain services wrapped as non-blocking task	✓ Done	88779820
8	Token validated: REST API confirmed OTF Signal Bot live	✓ Done	88779820
9	Bot invited to OTF Signals server (guild ID: 1500905...)	✓ Done	Manual
10	Railway Variables: DISCORD_BOT_TOKEN + TAOSTATS_API_KEY	✓ Done	Railway
11	Production deployment: Gateway connected, on_ready fired	✓ Done	Railway
12	E2E test: Discord msg → Gateway → Activity Log confirmed	✓ Done	Live
13	UI: Discord warning banner hidden when status = ok	✓ Done	88779820+

FILE CHANGE LOG

File	Changes
backend/services/signal_ingestor.py	Added discord import; _DISCORD_CHANNEL_KEYWORDS, _DISCORD_TARGET_CHANNELS, _discord_client/_discord_ta

backend/requirements.txt	Added discord.py>=2.3.2
backend/core/config.py	Added TAOSTATS_API_KEY: str = '', PERPLEXITY_API_KEY: str = '', DISCORD_BOT_TOKEN: str = '' to Settings model (pre
backend/main.py	signal_ingestor.start_all() moved to TOP of _boot_services() before any chain I/O; bittensor boot probe removed (substrate-int
backend/.env	DISCORD_BOT_TOKEN added (git-ignored, local only)
frontend/src/pages/ActivityLog.tsx	Discord warning banner condition changed from {isDiscord && ...} to {isDiscord && feed.status !== 'ok' && ...}; toggle disabled

DISCORD GATEWAY ARCHITECTURE

Message Flow

`Discord Server → WebSocket Gateway → discord.py on_message() → _message_is_relevant() keyword filter → push_event(category='signal') → Activity Log SIGNAL stream`

Component	Detail
Library	discord.py 2.7.1 (installed in production venv)
Connection	discord.Client with Intents.default() + message_content=True + messages=True
Token format	Static bot token (Bot prefix in Authorization header)
Keyword filter	subnet · announce · alpha · governance · validator · weight · staking · bittensor · tao · signal
Channel scope	_DISCORD_TARGET_CHANNELS = set() → reads all channels bot can see
Author filter	message.author.bot → skip (ignores other bots)
Reconnect	Exponential backoff: 5/15/30/60/120s; LoginFailure → 300s hold
on_ready	Sets feed enabled=True, status=ok, error=None, last_fetch=now()
on_message	Filters → push_event with channel, author, snippet, message_id, guild
on_disconnect	Sets feed status=connecting (reconnect loop handles re-entry)
Bot name	OTF Signal Bot #8669 (ID: 1500891557312594060)
Guild	OTF Signals (ID: 1500905975107031155)
Token verification	REST GET /api/v10/users/@me confirmed bot identity pre-deploy
Guild verification	REST GET /api/v10/users/@me/guilds confirmed OTF Signals membership

BOOT ORDER FIX — ROOT CAUSE ANALYSIS

Problem

bittensor's substrate-interface library performs synchronous WebSocket I/O on the asyncio event loop thread. Even when wrapped in asyncio.wait_for() or create_task(), the C-extension socket operations block the entire event loop, preventing discord.py's pending Gateway WebSocket handshake from completing.

Evidence

Local log froze at 'bittensor | Enabling default logging' for 2+ minutes. discord.client logged 'logging in using static token' (HTTP to Discord API succeeded) but on_ready never fired. Price service (pure httpx/aiohttp) started fine — proving the event loop ran between discord.py start and bittensor block.

Fix

Before (blocking)	After (non-blocking)
`signal_ingestor.start_all()  ← LAST`	`signal_ingestor.start_all()  ← FIRST`
`await bittensor.get_chain_info()`	`# Finney probe REMOVED`
`await price_service.start()`	`await price_service.start()`
`await subnet_cache.start()  ← BLOCKS`	`_aio.create_task(_start_chain_services())`
`await cycle_service.start()`	`  # subnet/cycle/agent/promotion inside task`
`await agent_service.start()`	`  # runs concurrently, never blocks boot`
`await promotion_service.start()`	

SIGNAL FEED STATUS — POST SESSION XXI

Feed	Status	Auth	Interval	Notes
CoinGecko	Connected	None	60s	TAO/USD price, 24h change, volume
Reddit r/bittensor	Connected	None	5 min	Community sentiment, RSS
TaoDaily News	Connected	None	30 min	Ecosystem news RSS
Discord	Connected ✓	Bot token ✓	Real-time	OTF Signals server, keyword filter
Taostats API	Key saved	Raw key ✓	60s	Toggle on to activate; key masked
Perplexity Sonar	Disabled	Pending	15 min	Awaiting API key from user

SESSION XXI COMMIT LOG

Hash	Message
d0081eda	feat: Discord Gateway live connector (discord.py 2.x, auto-reconnect, keyword filter)
88779820	fix: Discord Gateway + boot order — signal feeds first, chain services non-blocking · Settings: add TAOSTATS/PERPLEXITY/DISCORD key fields to Pydantic model · Boot: signal_ingestor.start_all() before any chain I/O · Boot: bittensor probe removed; chain services as fire-and-forget task · Token verified: OTF Signal Bot in OTF Signals guild

PENDING / CARRY-FORWARD

Item	Priority	Notes
Message Content Intent	High	Discord Dev Portal → Bot → Privileged Gateway Intents → Message Content Intent → ON. Without this, on_message receive
Perplexity API key	Medium	User deferred. Get at perplexity.ai/api · sonar model · ~$16/month at 15-min interval
Taostats API toggle	Low	Key saved + masked. Toggle ON in Signal Feeds drawer to activate 60s polling.
OTF Bittensor server invite	Low	Bot is in OTF Signals (personal). Main discord.gg/bittensor requires server admin invite.

DATABASE_URL typo	Low	Railway env var 'DATABASE_UR' missing L — carried from Session XVIII.
Wallet 0.227τ verification	Low	Verify on Taostats explorer — carried from Session XVIII.
Win rate progression	Ongoing	Monitor strategies toward 55% WR threshold for APPROVED_FOR_LIVE promotion gate.
SN65–128 subnet names	Ongoing	SUBNET_META placeholders — populate as subnets establish identities.

LESSONS LEARNED

1. bittensor substrate-interface blocks the event loop at the C-extension level

asyncio.wait_for(), create_task(), and any Python-level async wrapper cannot prevent C-extension socket calls from blocking the event loop thread. The only safe patterns are: (a) run_in_executor with a ThreadPoolExecutor, or (b) restructure boot order so blocking services start after non-blocking ones have had a chance to complete their async I/O. We chose (b) as the simpler, more maintainable solution.

2. Pydantic BaseSettings with extra='forbid' rejects unknown env vars

Adding env vars to Railway/local .env without declaring them in the Settings model causes ValidationError at startup. Any new env var must have a corresponding field in Settings before it can be used. Pattern: optional str fields with empty-string defaults.

3. discord.py on_ready is the authoritative connection signal

'Shard ID None has connected to Gateway' in discord.py's internal logger is the WebSocket handshake confirmation — on_ready fires immediately after. Using this log line to confirm production connection is reliable. The REST API /users/@me endpoint also provides a lightweight pre-flight token validity check without starting the full Gateway.

4. Token exposure in chat — safe here, regenerate if exposed publicly

The Discord bot token was shared in the private session chat to wire it into the backend. Tokens shared in private AI sessions are not at risk, but the same token should never be pasted into a public Discord channel, GitHub commit, or public log. If exposed publicly, immediately Reset Token in the Discord Developer Portal.

Session XXI · The Discord Gateway Sprint · Autonomous Trade Bot · May 4 2026 · All commits pushed to origin/main · 4/6 signal feeds connected · Gateway confirmed live